

Software Engineering

GitFlow

GitFlow

What is GitFlow?

- GitFlow is a branching model designed to structure and streamline software development
- It defines a clear workflow for managing code versions, ensuring stability, and facilitating collaboration in a team
- Introduced by Vincent Driessen (2010), it is widely used in Agile software development

What is GitFlow?

- GitFlow is a model, not a strict rulebook
- The core concepts remain the same, but companies adapt it
- Common Variations
 - Classic GitFlow (with *release* branches)
 - Simplified GitFlow (no *release* branches)
 - Trunk-Based Development (no *dev*, everything on *main*)
- The best GitFlow depends on your team's workflow and deployment strategy

Why should we use GitFlow?

- Separates stable code from ongoing development, reducing risks
- Allows multiple features to be developed in parallel on isolated branches, increasing efficiency
- Ensures quality and stability by integrating testing before release
- Facilitates collaboration using Merge Requests for code review
- Keeps track of releases using tags and structured versioning

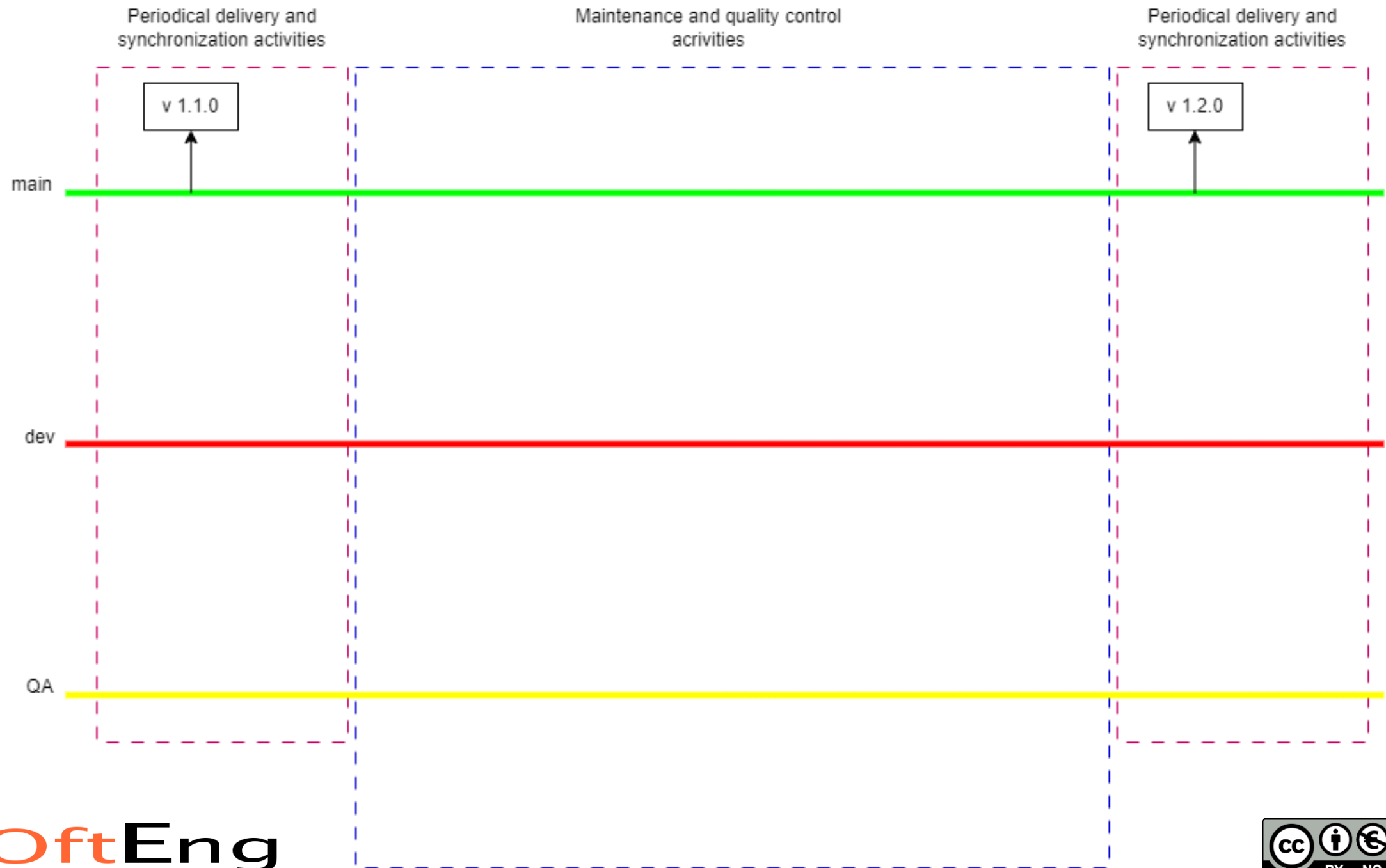
When should we use GitFlow?

- ALWAYS! (or at least in structured projects)
- Especially beneficial for projects with periodic releases (e.g., Agile methodologies)
- Essential for teams working in CI/CD environments, where automated builds and deployments rely on a structured branching model

GitFlow – A real-life example

Goal

- Periodic delivery of a new product version, ensuring predictable releases
- Maintaining high-quality standards, with a structured Quality Assurance process
- Enabling parallel work between development and Quality Assurance, avoiding bottlenecks
- Isolating production code from development and testing, allowing
 - Safe production bug fixes without interfering with ongoing development
 - A stable environment for Quality Assurance to validate before releasing new versions



Branch structure – persistent branches

- These branches must remain stable and serve as reference points for the production process.
- *main* branch
 - Production-ready code
- *dev* branch
 - Base for new features
- *QA* branch
 - Testing and integration branch

Persistent branches – *main* branch

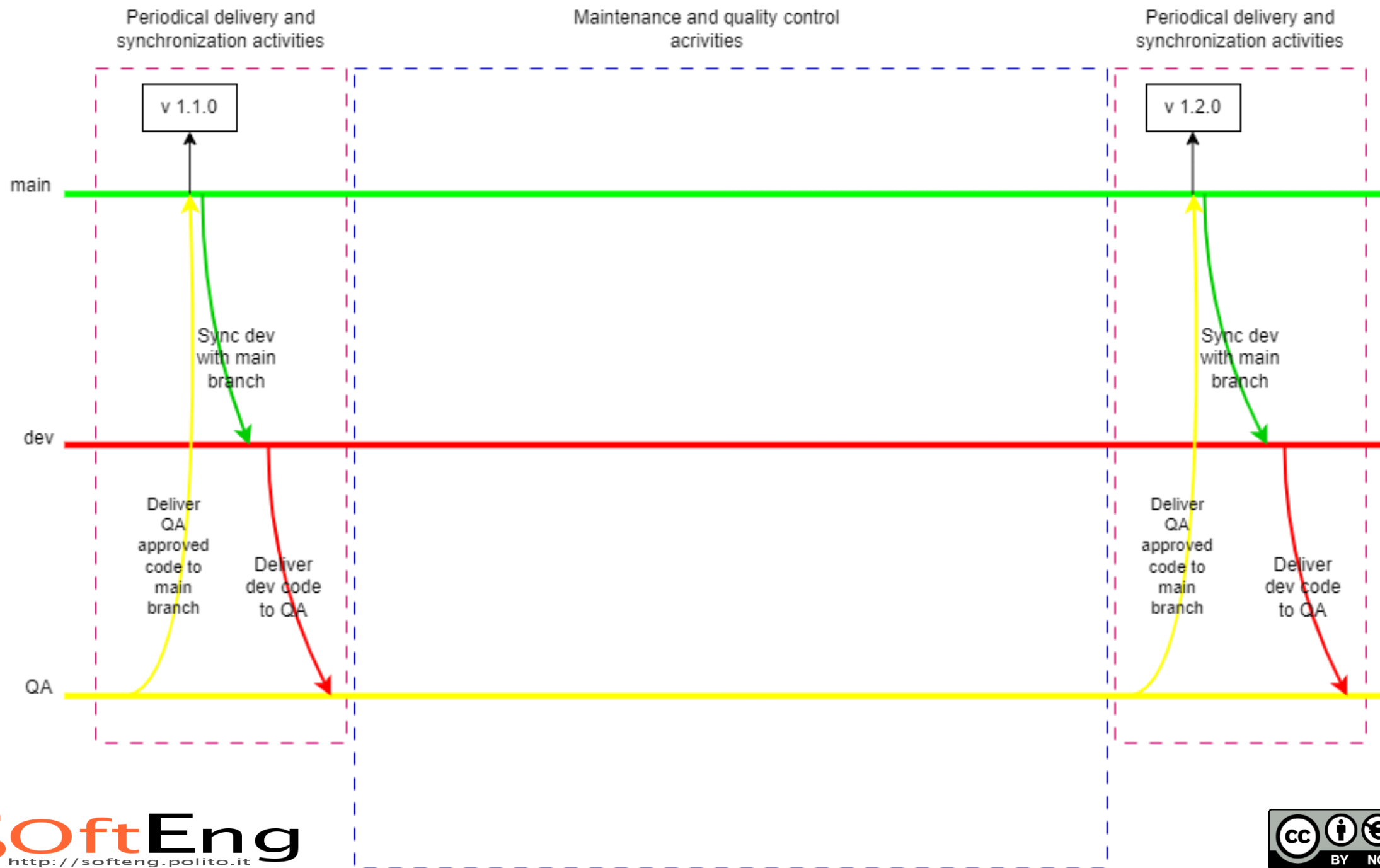
- Contains the latest stable and production-ready code
- Only approved, tested code is merged here
- Releases are marked using tags, such as v1.0.0
- Automated CI/CD pipelines trigger builds and deployments when code is merged
- Fixes for production bugs are handled using *hotfix*/ branches

Persistent branches– *dev* branch

- The main working branch where new features are integrated before testing
- Developers create *feature/* branches from *dev*
- *feature/* branches are merged into *dev* after development and unit testing
- Periodically, *dev* is merged into *QA* for integration testing

Persistent branches– QA branch

- QA branch is used for quality assurance and integration testing
- The Quality Assurance team performs
 - Regression testing to ensure existing features work
 - Integration testing to verify new code does not break the system
- If a bug is found, a *bugfix*/ branch is created to address it
- Once testing is complete, QA is merged into *main*, and a new release tag is created



Branch structure – ephemeral branches

- We avoid commits on persistent branches (*main, dev, QA*)
- The use of ephemeral branches isolates every single development in its branch avoiding any impact on other branches
- The use of ephemeral branches allows to create merge requests
 - Enable code reviews before merging
 - Maintain clean and structured code
 - Ensure that only validated changes reach persistent branches
- Three types of ephemeral branches
 - *feature/*
 - *hotfix/*
 - *bugfix/*

Ephemeral branches – *feature* branches

- Develop new functionalities before merging into *dev*
- Branch Naming Convention
 - *feature/#issueID-featureDescription*
 - *issueID* is the GitLab/Jira/GitHub... ID of the issue associated to the new feature
- Workflow
 - Branch is created from *dev*
 - Code is developed, including unit/integration tests
 - Merge Request is opened for team review and approval
 - Once approved, the branch is merged back into *dev* and deleted

Ephemeral branches – *hotfix* branches

- Fix critical bugs found in production (*main*)
- Branch Naming Convention
 - *hotfix/#issueID-bugDescription*
 - *issueID* is the GitLab/Jira/GitHub... ID of the issue associated to the bug
- Workflow
 - Branch is created from *main*
 - Bug is fixed and tested
 - Merge Request is opened for team review and approval
 - Once approved, the branch is merged back into *main* and deleted
 - A new tag and release is created on *main* branch
 - If necessary, the fix is cherry-picked into *dev* and *QA*

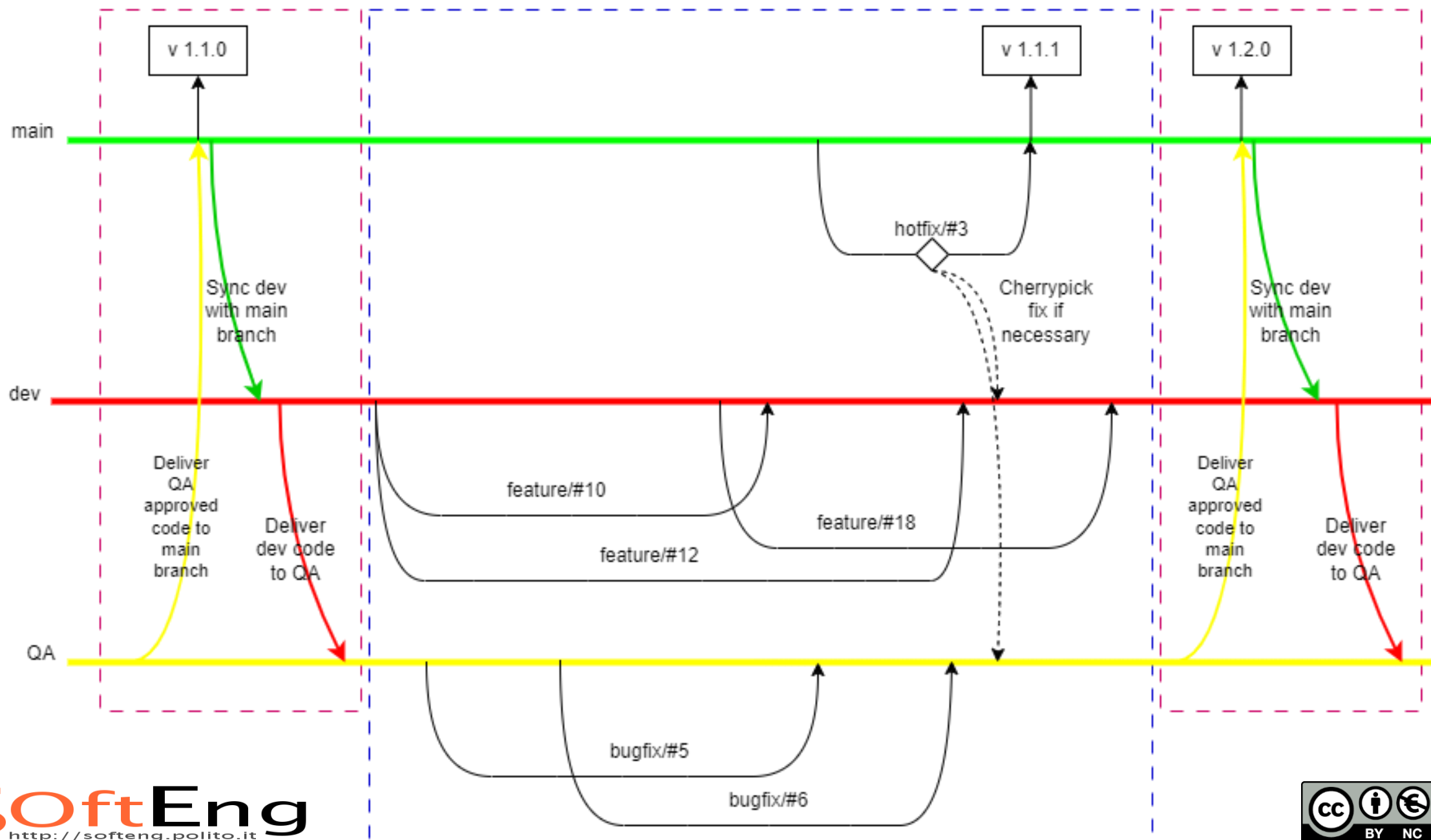
Ephemeral branches – *bugfix* branches

- Fix bugs found during testing (QA branch)
- Branch Naming Convention
 - *bugfix/#issueID-bugDescription*
 - *issueID* is the GitLab/Jira/GitHub... ID of the issue associated to the bug
- Workflow
 - Branch is created from QA when Quality Assurance team finds a bug
 - Bug is resolved and tested
 - Merge Request is opened for team review and approval
 - Once approved, the branch is merged back into QA and deleted
 - If necessary, it is also cherrypicked to *dev*

Periodical delivery and
synchronization activities

Maintenance and quality control
activities

Periodical delivery and
synchronization activities



GitFlow – best practices

- Keep *main* stable
 - Only merge QA-approved code
- Use meaningful Merge Requests
 - Provide clear descriptions and references, add images if GUI is impacted
- Write unit tests before merging to *dev*
- Keep *dev* in sync
 - Merge from *main* after releases
- Follow a strict naming convention for branches
- Use CI/CD automation
 - Trigger builds and tests on merges to main

GitFlow – bad practices to avoid

- Working directly on persistent branches
 -  Modifying *main*, *dev*, or *QA* without a separate ephemeral *hotfix/*, *feature/* or *bugfix/* branch
 -  Always use ephemeral branches and create Merge Requests
- Not syncing branches regularly
 -  Leaving *dev* outdated after a production bug fix
 -  Regularly merge *main* into *dev* after releases
- Not using meaningful branch names
 -  Using *feature/experiment1* instead of *feature/1234-add-login-form*
 -  Stick to the naming conventions (*feature/#issueld-featureDescription*)

GitFlow – bad practices to avoid

- Forgetting to delete merged branches
 -  Leaving old feature branches that clutter the repo
 -  Always delete ephemeral branches after merging
- Creating personal branches in the shared repo
 -  Using *johns-branch*, *my-test*, *fix-quick*, or similar personal names
 -  Branches in the shared repository must follow team conventions and be accessible to all
 -  If personal work is needed, use a personal fork and keep it synced

GitFlow – Key takeaways

- GitFlow provides structure for (especially Agile) development with well-defined branch roles
- Persistent branches (*main*, *dev*, *QA*) ensure stability and separation of concerns
- Ephemeral branches (*feature/*, *hotfix/*, *bugfix/*) keep work isolated and reviewable
- Merge Requests are essential for code reviews and quality control
- Synchronization between branches ensures all fixes and features reach production